Họ và tên: Nguyễn Đăng Thới Toàn MSHV: 25C15062

Báo cáo Bài tập thực hành 3 - Lập trình cho Trí tuệ Nhân tạo nâng cao

0. Tóm tắt báo cáo:

Bài 3 - Instance Segmentation trên dataset TACO: Sử dụng YOLO26n-seg và YOLO26s-seg để phân đoạn 10 nhóm rác thải. Trên tập test (ảnh đầu vào 640x640) YOLO26s đạt Mask mAP@0.5:0.95 bằng 21.20% cao hơn so với YOLO26n (17.27%). Với ảnh đầu vào 960x960, kết quả lần lượt đạt 27.91% và 22.87%. Bài làm cũng thực hiện mở rộng bằng thử nghiệm Data Efficiency Study và chuyển mô hình sang ONNX FP32 để đánh giá inference latency trên CPU.

1. Quá trình thực hiện

Yêu cầu ban đầu của bài tập là triển khai Instance Segmentation bằng Mask R-CNN (MMDetection). Tuy nhiên, trong quá trình thiết lập môi trường, em gặp khó khăn về tính tương thích giữa các phiên bản thư viện MMCV, MMDetection và môi trường PyTorch/CUDA đang sử dụng. Đề bài cho phép mở rộng hoặc thay thế bằng thuật toán hiện đại hơn cùng nhóm, em lựa chọn YOLO26n-seg và YOLO26s-seg thuộc Ultralytics để thực hiện bài toán Instance Segmentation trên bộ dữ liệu TACO.

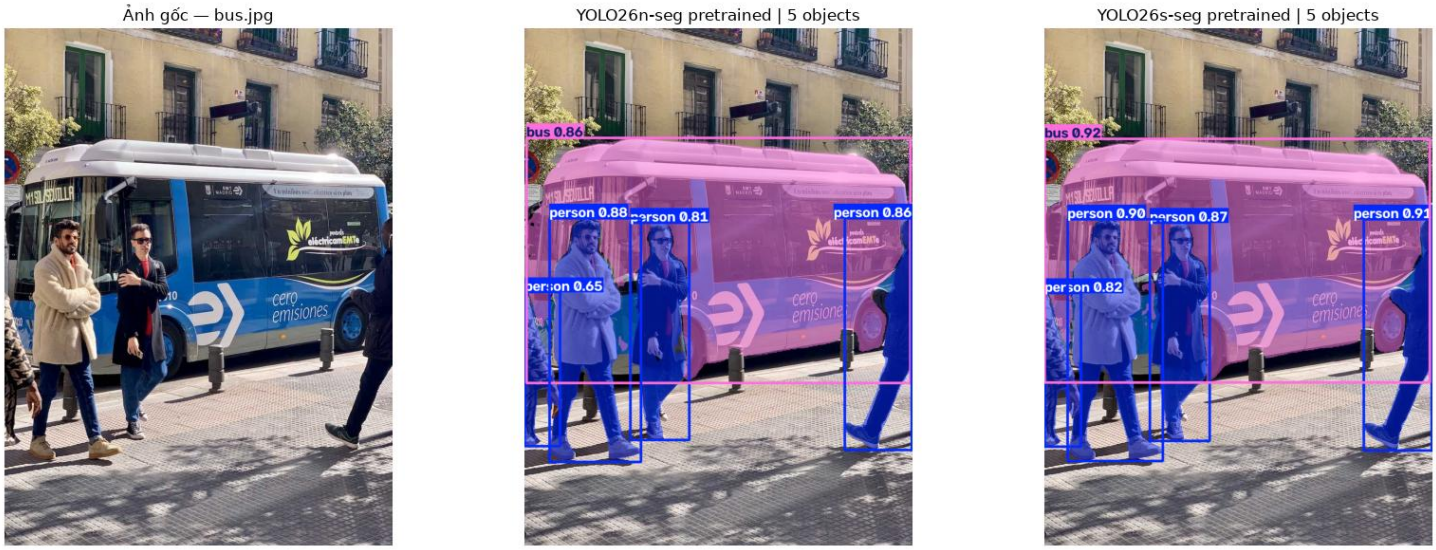
Các bước thực hiện chính:

- Chuẩn bị dữ liệu: Chuyển bộ dữ liệu TACO từ 60 lớp về 10 nhóm rác, gồm 1.500 ảnh và 4.784 đối tượng. Chia dữ liệu thành 1.200 ảnh train, 150 ảnh validation và 150 ảnh test.
- Phân tích dữ liệu (EDA): Trực quan hóa phân bố lớp, số lượng đối tượng trên mỗi ảnh và các polygon segmentation.
- Fine-tuning: Huấn luyện YOLO26n-seg và YOLO26s-seg trên GPU RTX 5090, với ảnh đầu vào 640x640, batch size 8, tối đa 200 epochs và Early Stopping.
- Đánh giá mô hình: Sử dụng Mask mAP@0.5 và Mask mAP@0.5:0.95, đồng thời phân tích AP theo từng lớp và trực quan hóa các trường hợp dự đoán tốt, kém.
- Data Efficiency Study: Thử nghiệm YOLO26n với 25%, 50% và 100% dữ liệu huấn luyện để đánh giá ảnh hưởng của lượng dữ liệu.
- ONNX Deployment: Export hai mô hình sang ONNX FP32, kiểm tra sự thay đổi độ chính xác và benchmark thời gian suy luận trên CPU.

Các thí nghiệm sử dụng cùng cách chia dữ liệu và random seed 42 để đảm bảo tính nhất quán khi so sánh.

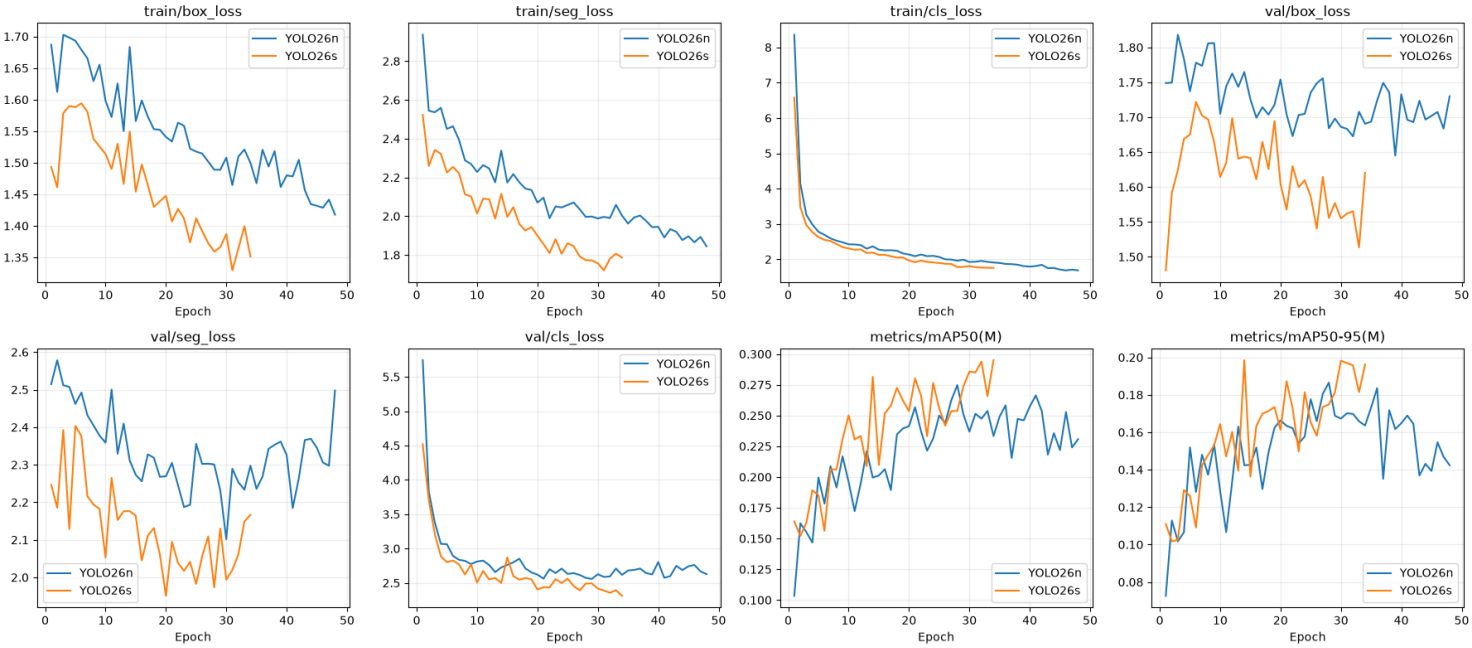
2. Kết quả đạt được

- *Inference với model pretrain trên một ví dụ có cùng domain với dataset COCO:*



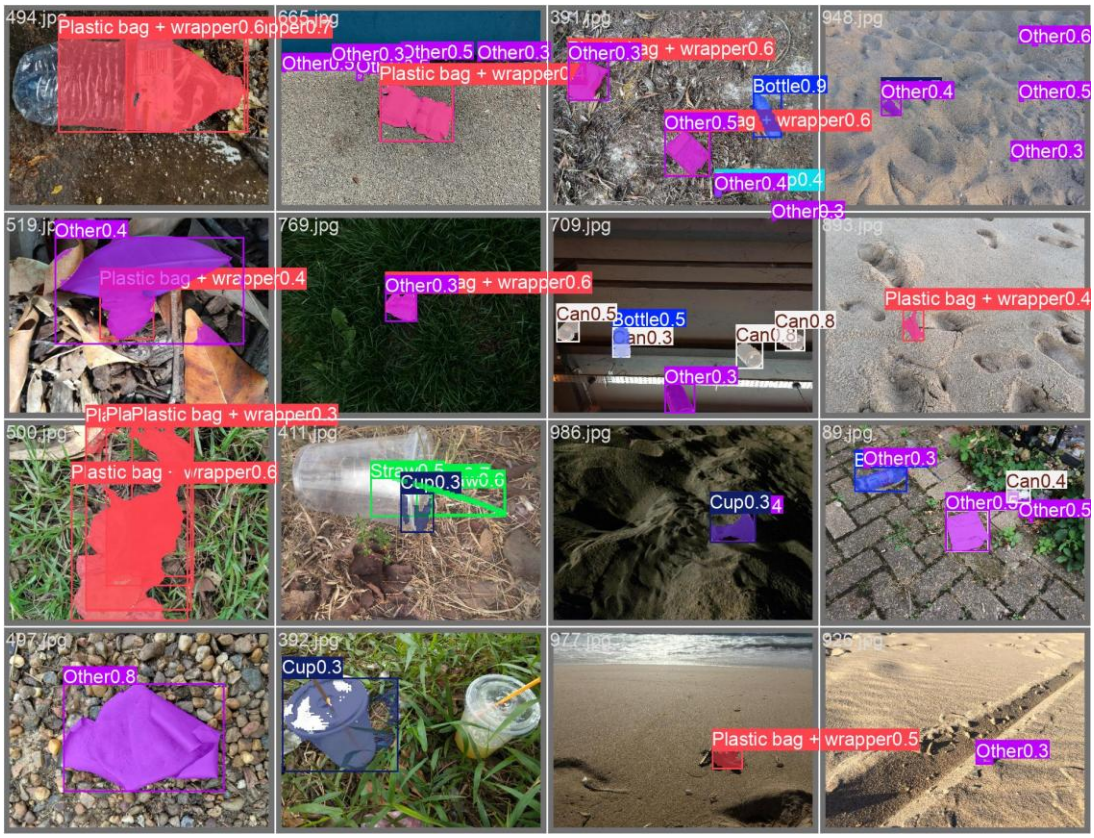
Hình 1. Kết quả inference với 1 ảnh demo có cùng domain với dataset COCO.

- Fine-tuning: YOLO26n dừng sau 48 epoch, YOLO26s dừng sau 34 epoch (cả 2 đều có Early Stopping).



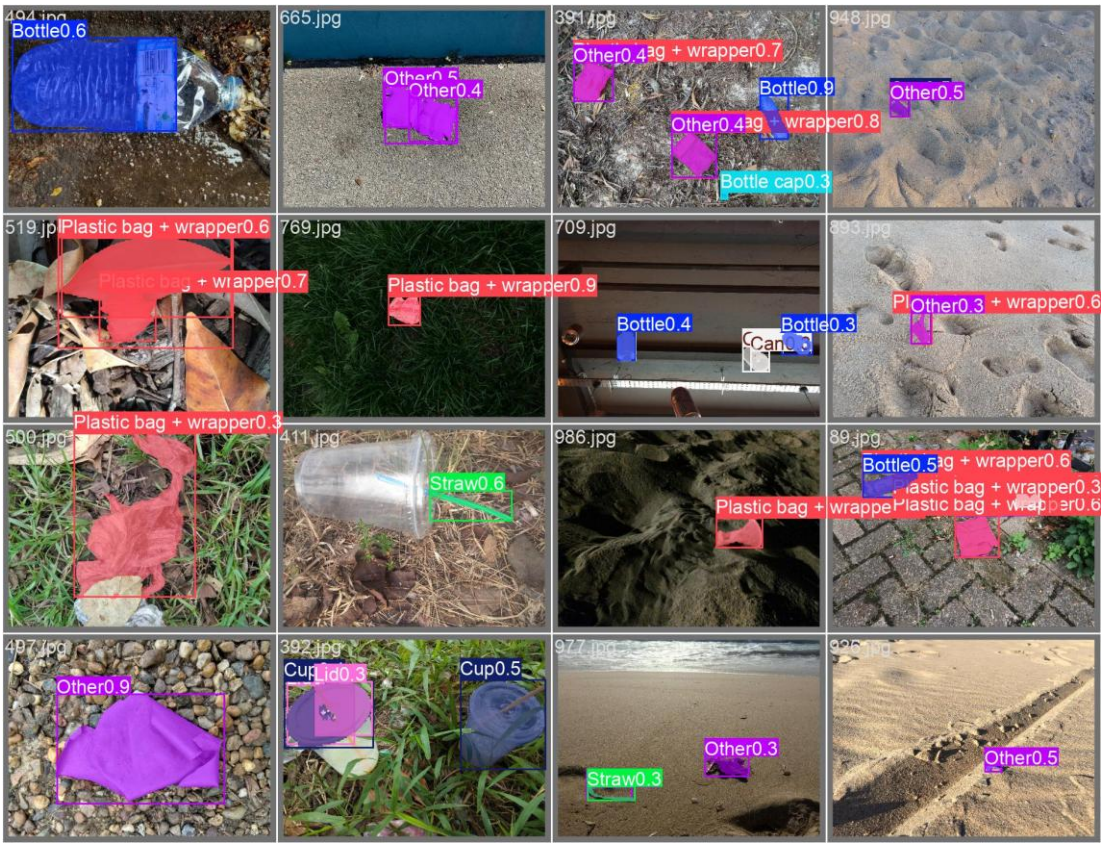
Hình 2. Quá trình hội tụ của YOLO26n và YOLO26s.

- Kết quả inference:
  - YOLO26n:



Hình 3. Kết quả inference của YOLO26n.

- YOLO26s:



Hình 4. Kết quả inference của YOLO26s.

- Segmentation Accuracy:
  - Kết quả trên tập validation:

	Box mAP50	Box mAP50-95	Mask mAP50	Mask mAP50-95
YOLO26n PyTorch	0.2730	0.1981	0.2582	0.1692
YOLO26s PyTorch	0.2941	0.2205	0.2799	0.1940

Hình 5. Kết quả trên tập validation.

  - Kết quả trên tập validation với từng nhãn:

	YOLO26n PyTorch	YOLO26s PyTorch
Bottle	0.3824	0.3940
Bottle cap	0.1759	0.1182
Can	0.2450	0.4910
Cigarette	0.0191	0.0413
Cup	0.1394	0.2275
Lid	0.3997	0.3007
Plastic bag + wrapper	0.1456	0.1836
Pop tab	0.0015	0.0013
Straw	0.0376	0.0625
Other	0.1463	0.1202

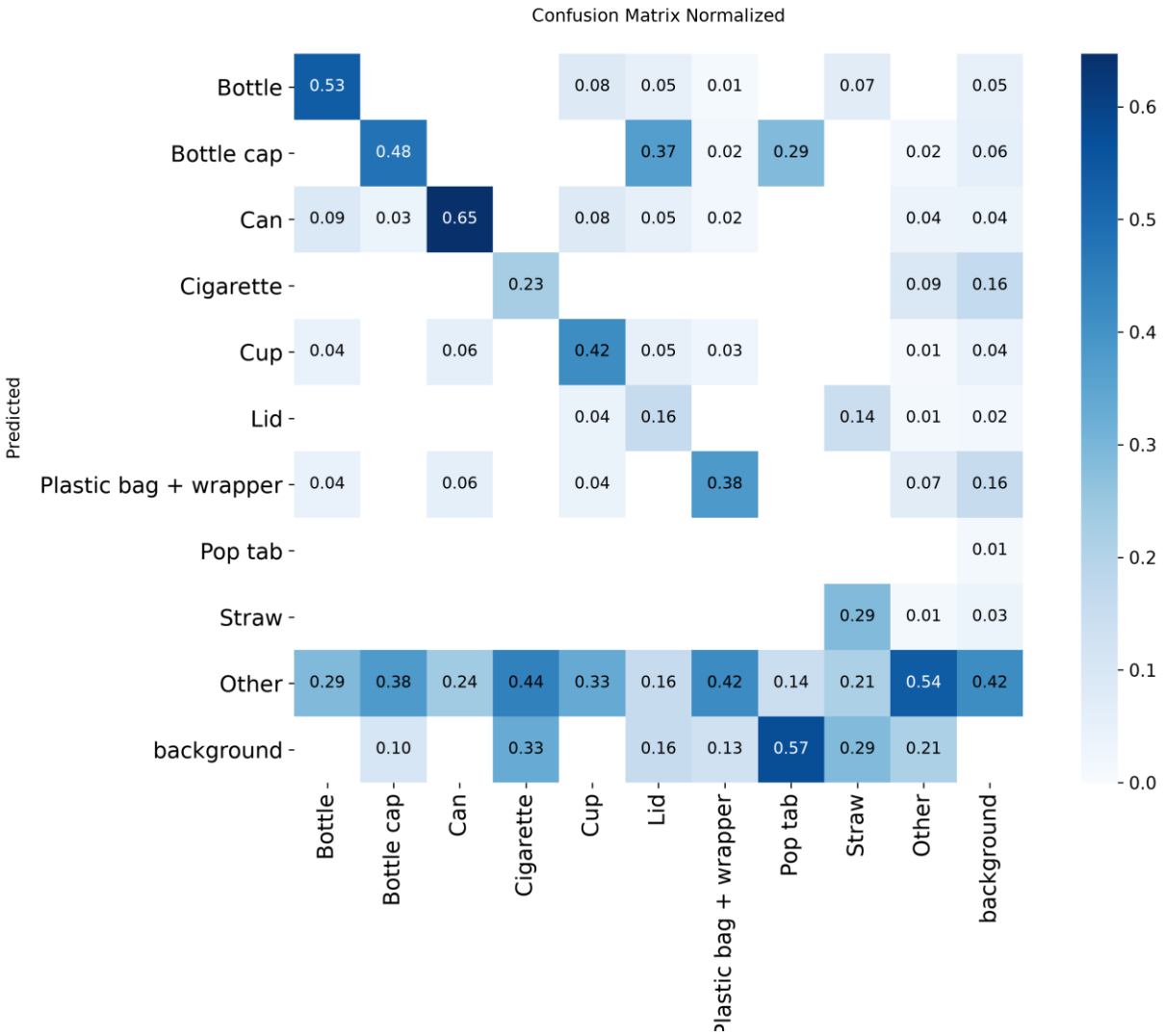
Hình 6. Kết quả trên tập validation với từng nhãn.

  - Kết quả trên tập test:

	Box mAP50-95	Mask mAP50	Mask mAP50-95
YOLO26n	0.1881	0.2653	0.1727
YOLO26s	0.2335	0.3127	0.2120

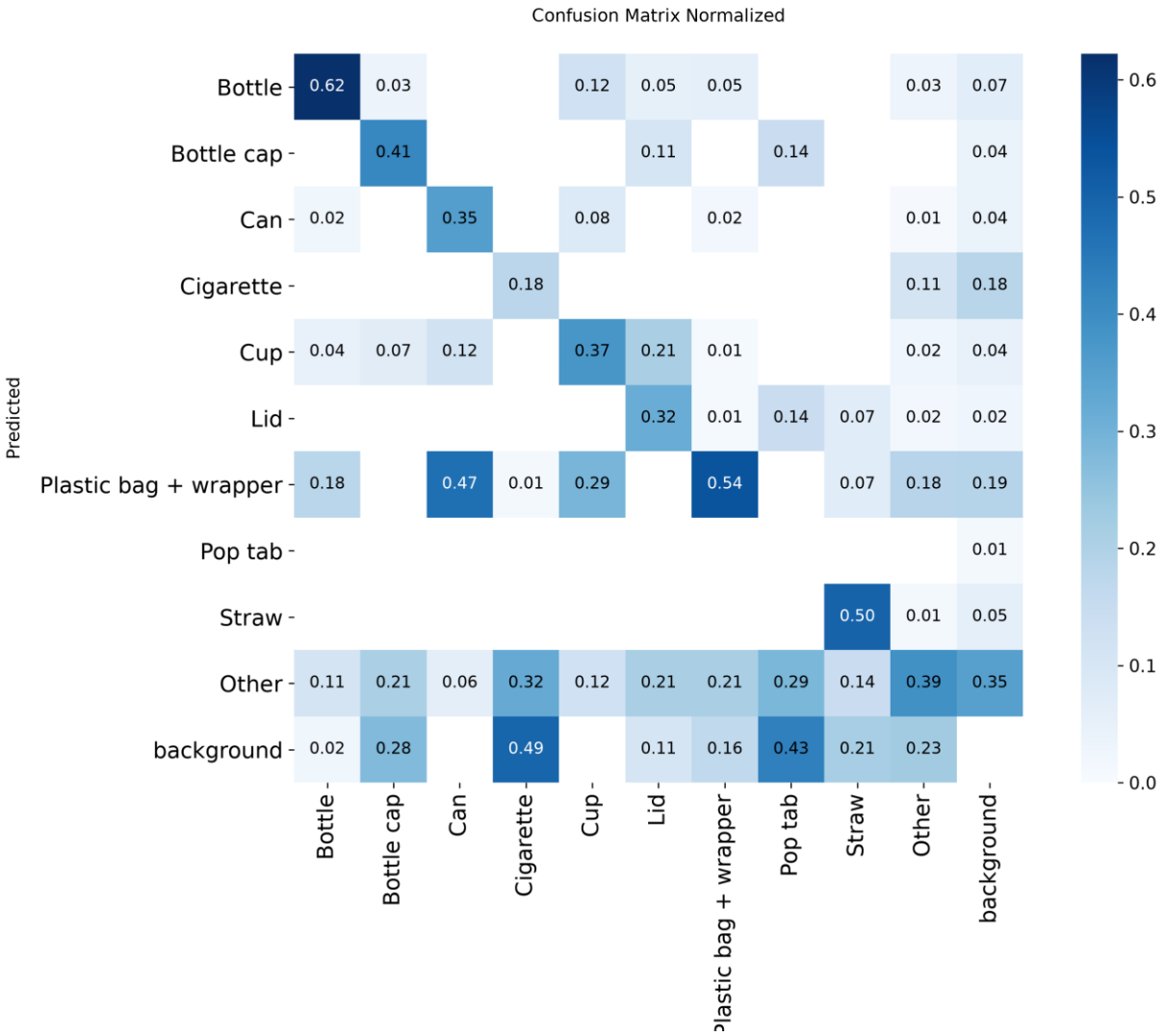
Hình 7. Kết quả trên tập test.

  - Ngoài ra, em cũng train với kích thước input lớn hơn là 960x960 ở Kaggle. Kết quả Mask mAP@50:95 của YOLO26n và YOLO26s lần lượt là 0.2287 và 0.2791. Notebook ở Kaggle tại đây.
- Error Analysis:
  - YOLO26n:



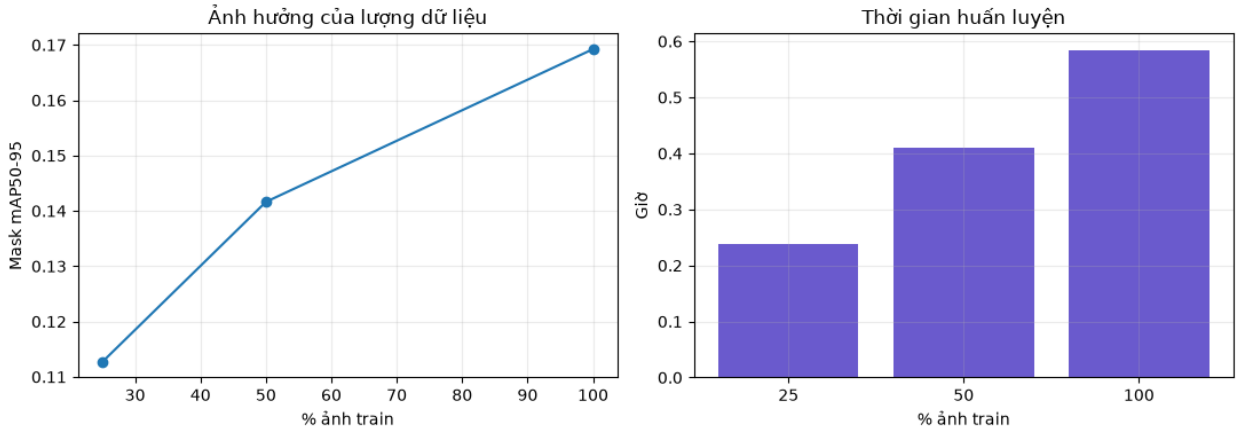
Hình 8. Confusion Matrix của YOLO26n.

- YOLO26s:



Hình 9. Confusion Matrix của YOLO26s.

- Data Efficiency Study: Khi tăng dữ liệu huấn luyện YOLO26n từ 25% lên 100%, Mask mAP@0.5:0.95 cải thiện từ 11.3% lên 16.9% (validation). Kết quả này cho thấy trong thiết lập thực nghiệm, việc bổ sung dữ liệu giúp nâng cao chất lượng của model.



Hình 10. Ảnh hưởng của lượng dữ liệu với chất lượng model.

- ONNX Deployment**:** Cả hai mô hình được export thành công sang ONNX FP32 và giữ được độ chính xác gần tương đương PyTorch trên validation. Tuy nhiên, benchmark trên CPU AMD Ryzen 9 9950X cho thấy ONNX Runtime chậm hơn PyTorch trong môi trường thử nghiệm:

Variant	count	mean_ms	std_ms	median_ms	p95_ms	FPS_sequential_est
YOLO26n PyTorch	250	13.78	0.61	13.82	14.64	72.59
YOLO26n ONNX FP32	250	34.58	14.52	34.79	59.04	28.92
YOLO26s PyTorch	250	29.15	1.08	29.13	30.12	34.30
YOLO26s ONNX FP32	250	71.55	23.42	76.79	107.39	13.98

Hình 11. So sánh Inference Latency giữa ONNX và Pytorch (CPU).

3. Khó khăn và khắc phục

- Mất cân bằng dữ liệu: TACO có phân bố lớp không đồng đều, chẳng hạn nhóm Other có 1.727 đối tượng nhưng Lid chỉ có 87. Em sử dụng AP theo từng lớp và phân tích các trường hợp dự đoán sai để đánh giá rõ hơn hạn chế của mô hình.
- Dữ liệu huấn luyện hạn chế: TACO chỉ có 1.500 ảnh, trong khi nhiều đối tượng rác nhỏ và có hình dạng phức tạp. Em thử nghiệm các tỷ lệ dữ liệu 25%, 50% và 100% để phân tích ảnh hưởng của kích thước tập huấn luyện.
- Hiệu năng khi triển khai ONNX: Mặc dù chuyển đổi ONNX thành công và gần như không làm giảm accuracy, thời gian suy luận trên CPU lại cao hơn PyTorch. Em thực hiện benchmark trên cùng tập ảnh, cùng cấu hình đầu vào và tách riêng quá trình đo cho từng mô hình để so sánh.

4. Nhận xét và đánh giá

- Qua thử nghiệm, em nhận thấy YOLO26s cho chất lượng segmentation tốt hơn YOLO26n, nhưng có kích thước checkpoint lớn hơn đáng kể (23,33 MB so với 6,59 MB). Do đó, việc lựa chọn mô hình cần cân nhắc giữa chất lượng dự đoán và tài nguyên triển khai.
- YOLO26s cải thiện khả năng nhận diện các lớp Straw (29% → 50%), Lid (16% → 32%) và Plastic bag + wrapper (38% → 54%). YOLO26s suy giảm ở các lớp Can (65% → 35%), Other (54% → 39%) và Bottle cap (48% → 41%). Các giá trị % được đề cập ở Confusion Matrix, không phải là mAP.
- Cả hai mô hình đều gặp khó khăn với Pop tab (gần 0% nhận diện đúng) và có xu hướng nhầm lẫn nhiều đối tượng thành Other. YOLO26s còn nhầm 47% đối tượng Can thành Plastic bag + wrapper, so với 6% ở YOLO26n (đây là tỷ lệ trên Confusion Matrix đã chuẩn hóa, không phải mAP).
- Ngoài ra, lượng dữ liệu huấn luyện có ảnh hưởng rõ rệt đến kết quả. Khi tăng số ảnh từ 300 lên 1.200, Mask mAP@0.5:0.95 tăng khoảng 5,6 điểm phần trăm. Tuy nhiên, kết quả phân đoạn vẫn còn hạn chế đối với các lớp có ít mẫu hoặc đối tượng nhỏ.
- Một điểm em thấy đáng chú ý là ONNX không nhất thiết giúp tăng tốc inference. Trong môi trường CPU được thử nghiệm, ONNX FP32 chậm hơn PyTorch dù độ chính xác gần như không thay đổi.

5. Thảo luận mở rộng

- Ngoài yêu cầu cơ bản, em đã mở rộng bài tập bằng cách so sánh YOLO26n và YOLO26s, phân tích lỗi theo từng lớp, đánh giá ảnh hưởng của lượng dữ liệu huấn luyện và thử nghiệm triển khai ONNX FP32 trên CPU.

- Kết quả cho thấy mô hình lớn hơn có thể cải thiện độ chính xác, nhưng không giải quyết hoàn toàn khó khăn từ dữ liệu ít và mất cân bằng lớp. Đồng thời, việc chuyển đổi mô hình sang định dạng khác không đảm bảo cải thiện tốc độ nếu chưa tối ưu cho phần cứng và runtime thực tế.

6. Nguồn tham khảo

- Ultralytics, *YOLO Instance Segmentation Documentation*. https://docs.ultralytics.com/tasks/segment/
- Proença et al. (2020), *TACO: Trash Annotations in Context*. https://github.com/pedropro/TACO
- Ultralytics, *Model Training and TensorBoard Integration*. https://docs.ultralytics.com/modes/train/
- ONNX Runtime, *Official Documentation*. https://onnxruntime.ai/docs/